



ENGINEER NOTES

ON-CALL ENGINEER HANDBOOK

From Production Alerts
to Incident Resolution



ALERT



INVESTIGATE



MITIGATE



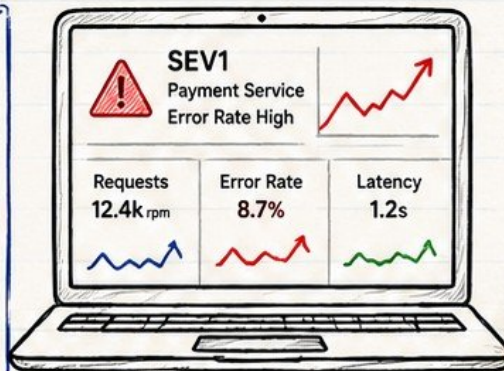
RECOVER



IMPROVE

WHAT'S INSIDE

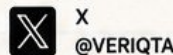
- ✓ Monitoring & Alerting
- ✓ Incident Response Process
- ✓ Troubleshooting Playbooks
- ✓ Mitigation & Recovery
- ✓ Communication & Escalation
- ✓ Post-Incident Review
- ✓ Best Practices & Checklists



ON-CALL PRINCIPLES

1. Protect customers first.
2. Stabilize the system.
3. Use evidence, not assumptions.
4. Change one thing at a time.
5. Learn and improve constantly.

YOUR RESPONSIBILITY. THEIR TRUST. OUR RELIABILITY.
YOU ARE THE CALM IN THE CHAOS.



1. THE ON-CALL ENGINEER



WHAT ON-CALL ENGINEERING MEANS

- On-call engineering is the practice of being on duty to respond to production issues.
- You are the first line of defense when things break.
- Your goal: restore service, reduce impact, and learn from every incident.



PRIMARY AND SECONDARY ON-CALL

- Primary on-call: first responder and incident owner.
- Secondary on-call: backup support and escalation.
- Both must be prepared, reachable, and ready to act.



RESPONSIBILITIES DURING A SHIFT

- Monitor alerts and system health.
- Acknowledge and assess incidents quickly.
- Investigate and mitigate issues.
- Communicate clearly and frequently.
- Escalate when required.
- Document actions and outcomes.



INCIDENT OWNERSHIP

- Own the incident from start to resolution.
- Ensure the right people are involved.
- Drive the incident to a stable state.
- Follow up until full recovery is confirmed.



OPERATIONAL READINESS

- Know your systems, dependencies, and runbooks.
- Ensure access to all required tools.
- Keep escalation contacts up to date.
- Stay informed about changes and deployments.
- Test your recovery and rollback procedures.



WHAT ON-CALL ENGINEERS SHOULD NOT DO

- Do not ignore alerts or delay acknowledgment.
- Do not make changes without understanding the impact.
- Do not restart or rollback blindly.
- Do not communicate without verification.
- Do not take shortcuts that create future problems.



PRODUCTION MINDSET

- Protect the customer experience first.
- Think before you act.
- Use evidence, not assumptions.
- Prefer safe, reversible actions.
- Learn, improve, and prevent recurrence.

YOUR GOAL



RESTORE SERVICE
REDUCE IMPACT
LEARN & IMPROVE

ON-CALL REALITY



ALERTS DON'T FOLLOW
BUSINESS HOURS.
BE PREPARED.
BE CALM. BE FOCUSED.
BE EFFECTIVE.

ON-CALL SUCCESS



OBSERVE



UNDERSTAND



ACT



COMMUNICATE



IMPROVE



On-call is more than technical skill. It is responsibility, judgment, communication, and a commitment to keep the system and the business running.

YOU ARE THE CALM IN THE CHAOS.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

2. PREPARING FOR AN ON-CALL SHIFT



ON-CALL HANDOVER

- Get a clear update from the outgoing engineer.
- Ask questions about open items.
- Confirm anything unclear.
- Acknowledge when you are ready.



REVIEWING ACTIVE INCIDENTS

- Check all ongoing incidents.
- Understand current status and impact.
- Review recent actions taken.
- Identify anything that needs follow up.



KNOWN PRODUCTION RISKS

- Review risk register or risk dashboard.
- Check systems with known instability.
- Identify dependencies at risk.
- Be aware of capacity constraints.



RECENT DEPLOYMENTS

- Review deployments in the last 24-72 hrs.
- Identify services and versions deployed.
- Check for canary or progressive rollouts.
- Know rollback plan if needed.



MAINTENANCE WINDOWS

- Check today's and upcoming windows.
- Understand expected impact.
- Verify change approvals.
- Monitor during maintenance.



ESCALATION CONTACTS

- Know who to escalate to and when.
- Keep on-call escalation tree handy.
- Check time zones for key contacts.
- Ensure contact methods work.



DASHBOARD & TOOLING ACCESS

- Verify access to all dashboards.
- Confirm alerting tools are working.
- Check log, metric, tracing systems.
- Ensure CLI, VPN, and accounts work.



TIP

Preparation reduces stress, speeds up response, and protects users.

PRE-SHIFT READINESS CHECKLIST

☐ ON-CALL HANDOVER COMPLETED

I have received a clear handover and understand all open items.

☐ ACTIVE INCIDENTS REVIEWED

I have reviewed all ongoing incidents and their current status.

☐ PRODUCTION RISKS REVIEWED

I am aware of known risks and unstable components.

☐ RECENT DEPLOYMENTS REVIEWED

I know what was deployed and any related risks.

☐ MAINTENANCE WINDOWS CHECKED

I have reviewed today's and upcoming maintenance windows.

☐ ESCALATION CONTACTS VERIFIED

I know who to contact and how to reach them.

☐ DASHBOARDS ACCESS VERIFIED

I can access all required dashboards and monitoring tools.

☐ TOOLING ACCESS VERIFIED

I can access logs, metrics, tracing, CLI, VPN, and accounts.

☐ ALERTING CHANNELS CHECKED

I am receiving alerts and notifications in all channels.

☐ I AM READY

I understand my responsibilities and am ready for the shift.



BE PREPARED. STAY FOCUSED. PROTECT OUR USERS.



PREPARATION IS THE FIRST STEP TO INCIDENT RESOLUTION.
START READY. STAY READY.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

3. MONITORING, ALERTS, AND SIGNALS

METRICS



- Numerical data measured over time.
- Show trends and changes.
- Example: CPU usage, request rate, latency.

LOGS



- Detailed records of what happened.
- Essential for deep investigation.
- Example: application logs, error logs.

TRACES



- Track requests as they flow through services.
- Identify latency and failures across hops.
- Example: distributed tracing (Jaeger, OpenTelemetry).

EVENTS



- Discrete occurrences in the system.
- Useful for state changes and alerts.
- Example: deployment started, instance down.

SYMPTOMS VERSUS CAUSES

SYMPTOM



- What you see.
- The effect.
- Example: High latency.

VS

CAUSE



- Why it is happening.
- The root reason.
- Example: Database connection pool exhausted.



ACTIONABLE ALERTS

- ✓ Clear and specific.
- ✓ Tied to user or business impact.
- ✓ Include context and next steps.
- ✓ Reduce time to acknowledge and act.



ALERT FATIGUE

- Too many alerts.
- Many are not important.
- Leads to ignored critical alerts.
- Causes stress and slow response.



SIGNAL-TO-NOISE RATIO

Good monitoring has high signal (meaningful alerts) and low noise (unnecessary alerts).

$$\text{High Quality Monitoring} = \frac{\text{High Signal}}{\text{Low Noise}}$$



GOLDEN SIGNALS (SRE)

Latency



How long requests take.

Traffic



How many requests you serve.

Errors



The rate of failed requests.

Saturation



How full your system resources are.

Availability



Is the service working for users?

RED METHOD (FOR REQUEST-BASED SYSTEMS)

Rate



Requests per second.

Errors



Error rate (4xx, 5xx).

Duration



Request duration.

Best for APIs, web apps, and services.

USE METHOD (FOR RESOURCE-BASED SYSTEMS)

Utilization



How busy the resource is.

Saturation



How full the resource is.

Errors



Error rate of the resource.

Discards



Dropped or discarded work.

Best for infrastructure, databases, queues, and hosts.



GOOD MONITORING TELLS YOU WHEN SOMETHING IS WRONG.
GREAT MONITORING HELPS YOU FIND OUT WHY AND WHAT TO DO NEXT.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

4. RECEIVING A PRODUCTION ALERT

A CALM, STRUCTURED RESPONSE IN THE FIRST MINUTES
SETS THE TONE FOR A FAST RESOLUTION.

1



ALERT ACKNOWLEDGMENT

- Acknowledge the alert immediately.
- This stops notification noise for others.
- It shows the incident is being handled.

2



INITIAL ALERT VALIDATION

- Check if the alert is real or a false positive.
- Verify the metric, threshold, and source.
- Check monitoring dashboard and logs briefly.

3



DETERMINE AFFECTED SERVICE

- Identify the service, component, or dependency.
- Understand what is failing.
- Note the environment and region.

4



CHECK CURRENT SYSTEM STATE

- Review dashboards, metrics, and logs.
- Check recent deployments or changes.
- Understand the current health of the system.

5



CORRELATE RELATED ALERTS

- Look for related alerts across services.
- Group noisy alerts to find the real signal.
- Identify the pattern and common factor.

6



IDENTIFY CUSTOMER IMPACT

- Is the service degraded or completely down?
- Who is affected (users, tenants, regions)?
- Understand the business impact.

7



ESTABLISH INCIDENT SEVERITY

- Use SEV1-SEV4 model.
- Consider impact, urgency, and blast radius.
- Set the initial severity (you can adjust later).

8



DECIDE WHETHER ESCALATION IS REQUIRED

- Do you have the context and skills to resolve this?
- Do you need more eyes or a specialist?
- Escalate early if the impact is growing.

QUICK CHECK

- ✓ Alert acknowledged
- ✓ Alert validated
- ✓ Affected service identified
- ✓ System state checked
- ✓ Related alerts correlated
- ✓ Customer impact identified
- ✓ Severity established
- ✓ Escalation decision made



SEVERITY GUIDE

SEV 1
CRITICAL



Major outage.
Severe impact.
All hands needed.

SEV 2
HIGH



Partial outage.
High impact.
Urgent response.

SEV 3
MEDIUM



Degraded service.
Limited impact.
Work during shift.

SEV 4
LOW



Minor issue.
Minimal impact.
Normal priority.

GOLDEN RULE



Verify before you act.
Understand before you change.
Communicate early and clearly.

FIRST MINUTES MATTER:

ACKNOWLEDGE → ASSESS → UNDERSTAND → DECIDE → ACT



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

5. INCIDENT SEVERITY AND PRIORITIZATION

SEV1 TO SEV4 MODELS

SEVERITY	CUSTOMER IMPACT	BUSINESS IMPACT	RESPONSE TIME GOAL	EXAMPLES
SEV 1 CRITICAL 	- Major outage. - Many or all users cannot use the service. - Complete service interruption.	- Critical business functions down. - Major financial, contractual, or reputation risk. - Executive visibility.	 15 MINUTES or less	- Entire service down for all users. - Payments not processed. - Data loss or corruption.
SEV 2 HIGH 	- Partial outage. - Significant number of users affected. - Important features not working.	- High business impact. - Workarounds exist but are painful. - Revenue or operations affected.	 1 HOUR or less	- Checkout fails for many users. - Performance severely degraded. - Critical feature unavailable.
SEV 3 MEDIUM 	- Some users affected. - Minor features not working. - Limited impact.	- Moderate business impact. - Not all operations affected. - Limited customer dissatisfaction.	 4 HOURS or less	- Non-critical feature broken. - Intermittent errors. - Performance slower than normal.
SEV 4 LOW 	- Minimal or no user impact. - Cosmetic or minor issues.	- Low business impact. - Does not affect core operations. - No customer impact.	 NEXT BUSINESS DAY	- Typo in UI. - Enhancement request. - Documentation update.

FACTORS USED TO DETERMINE SEVERITY

 CUSTOMER IMPACT How many users are affected and how badly?	 BUSINESS IMPACT What is the financial, operational, or reputation impact?	 BLAST RADIUS How many services, regions, or tenants are affected?	 AVAILABILITY DEGRADATION Is the service down, degraded, or unavailable?	 DATA INTEGRITY RISKS Is data at risk of loss, corruption, or inconsistency?	 SECURITY IMPLICATIONS Is there a security breach or exposure risk?
---	--	--	--	--	---



SEVERITY ESCALATION

- Escalate if the impact is worse than initially thought.
- Escalate if more services or users are affected.
- Escalate if there is risk to data, security, or compliance.
- Escalate if mitigation fails and the issue worsens.
- Escalate when executive or customer visibility increases.



SEVERITY DOWNGRADE

- Downgrade when the impact is reduced.
- Downgrade when mitigation is effective.
- Downgrade when fewer users or services are affected.
- Downgrade when the risk to data or security is removed.
- Communicate clearly when downgrading.



ACCURATE SEVERITY DRIVES THE RIGHT RESPONSE.
GET THE SEVERITY RIGHT. GET THE RESPONSE RIGHT.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

6. THE FIRST 15 MINUTES OF AN INCIDENT



WHAT YOU DO IN THE FIRST 15 MINUTES CAN DETERMINE THE OUTCOME OF THE INCIDENT.

1



ACKNOWLEDGE

- Acknowledge the alert immediately.
- This confirms you are aware and engaged.

2



ASSESS

- Validate the alert is real.
- Identify the affected service and symptoms.
- Check dashboards and logs quickly.

3



STABILIZE

- If the service is degrading or down, take immediate actions to stop the impact from getting worse.

4



ESTABLISH OWNERSHIP

- Take ownership of the incident.
- Avoid confusion about who is handling it.
- Update the incident tracker if available.

5



CREATE AN INCIDENT CHANNEL

- Open or join the incident channel.
- Keep communication in one place.
- Share what you know so far.

6



CHECK RECENT CHANGES

- Review deployments, configuration changes, feature flags, and infrastructure changes.
- Look at the last 1–2 hours.

7



GATHER EVIDENCE

- Collect logs, metrics, traces, and alerts.
- Capture timestamps and error messages.
- Save information before making changes.

8



AVOID PREMATURE ASSUMPTIONS

- Don't jump to conclusions.
- Confirm facts with real data.
- Stay calm and follow the process.

9



DECIDE: MITIGATE OR INVESTIGATE

- If the impact is high, mitigate first.
- If the impact is low or unknown, investigate.
- Reassess and adjust as new information comes in.

FIRST 15-MINUTE GOAL



Understand the situation, stop the bleeding, and create clarity.

KEEP THIS IN MIND

- ✓ Customers don't care about the cause, they care about the impact.
- ✓ Speed is important, but safe actions are critical.
- ✓ Clear communication reduces panic.
- ✓ Facts over assumptions.

WHAT TO AVOID

- ✗ Ignoring or delaying acknowledgment.
- ✗ Making changes before understanding the impact.
- ✗ Working in isolation.
- ✗ Sharing unverified information.
- ✗ Trying to fix everything at once.

GOOD HABITS



- ✓ Breathe. Stay calm.
- ✓ Follow the process.
- ✓ Communicate early.
- ✓ Document as you go.
- ✓ Think before you act.



THE FIRST 15 MINUTES SET THE DIRECTION.
CALM MIND. CLEAR THINKING. SMART ACTIONS.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

7. PRODUCTION TROUBLESHOOTING FRAMEWORK

A SYSTEMATIC APPROACH TO FIND THE CAUSE,
NOT JUST APPLY TEMPORARY FIXES.

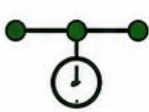
1



DEFINE THE SYMPTOM

- Be specific and observable.
- What is failing, where, and how?
- Avoid assumptions and broad statements.

2



ESTABLISH A TIMELINE

- When did the issue start?
- What happened just before?
- Use timestamps from logs, alerts, and users.

3



IDENTIFY WHAT CHANGED

- Deployments, configs, code, data, traffic?
- Any infrastructure or dependency changes?
- Even small changes can cause big issues.

4



DETERMINE BLAST RADIUS

- Who is affected (users, services, regions)?
- What features or integrations are impacted?
- How wide is the impact?

5



COMPARE HEALTHY AND UNHEALTHY SYSTEMS

- Find a working example to compare.
- Check metrics, configs, logs, and behavior.
- Look for differences, not similarities.

6



FOLLOW DEPENDENCIES

- Map the request path and dependencies.
- Check downstream and upstream services.
- Verify network, DNS, and external systems.

7



FORM HYPOTHESES

- Based on data, not guesses.
- List possible causes in priority order.
- Keep it short and testable.

8



TEST HYPOTHESES SAFELY

- Run safe tests in production.
- Use read-only checks and limited actions.
- Validate or eliminate quickly.

9



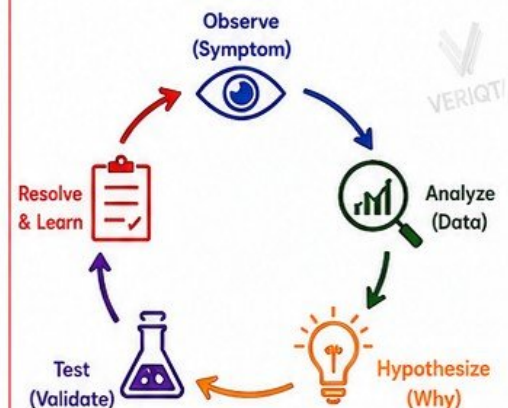
PRESERVE EVIDENCE

- Save logs, metrics, traces, and configs.
- Capture before you change anything.
- Document findings for post-incident review.

CORE PRINCIPLES

- ✓ Stay calm and follow the process.
- ✓ Use data, not opinions.
- ✓ Work from symptoms to cause.
- ✓ Test safely and deliberately.
- ✓ Communicate clearly and often.
- ✓ Document everything.

TROUBLESHOOTING CYCLE



ESSENTIAL TOOLKIT



MONITORING

Metrics, dashboards, alerts.



LOGS

Application, system, and access logs.



TRACES

Distributed traces, request flows.



CLI / SCRIPTS

Quick checks and validations.



RUNBOOKS

Reference steps, known issues.



FOLLOW THE FRAMEWORK. STAY CURIOUS.
FIND THE CAUSE. PREVENT THE NEXT INCIDENT.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA



8. LINUX AND HOST TROUBLESHOOTING



Goal: Quickly identify and diagnose common host-level issues that impact application performance and availability.

KEY AREAS TO CHECK



CPU SATURATION

High CPU usage can slow down or freeze processes.



MEMORY PRESSURE

Low available memory or high swap usage degrades performance.



DISK EXHAUSTION

Disk full (100%) prevents writes and causes application failures.



INODE EXHAUSTION

When inodes are used up, new files cannot be created even if disk space is free.



PROCESS FAILURES

Critical processes may crash, hang, or exit unexpectedly.



FILE DESCRIPTORS

Too many open files can cause service errors or new connections to fail.



SYSTEM LOAD

High load average indicates the system is overloaded.



NETWORK CONNECTIONS

Too many connections, stuck connections, or port issues impact services.



SERVICE FAILURES

System or application services may stop or fail to start.



KERNEL AND SYSTEM LOGS

Logs provide clues about hardware, kernel, and system-level problems.

ESSENTIAL COMMANDS

`$ top`

Real-time view of CPU, memory, and processes.

`$ free -m`

Display memory usage in MB.

`$ df -h`

Check disk space usage (human readable).

`$ df -i`

Check inode usage on filesystems.

`$ ps aux`

List all running processes.

`$ systemctl status`

Check status of a system service.

`$ journalctl`

View and filter systemd logs.

`$ ss -tulpn`

Show network connections and listening ports.

`$ lsof`

List open files and processes using them.

`$ dmesg`

Display kernel ring buffer messages.



TIP: Always collect evidence before making changes. Check recent changes, correlate with alerts, and isolate the root cause.





9. NETWORK AND DNS INCIDENTS



Goal: Quickly diagnose connectivity, routing, DNS, and transport issues that block access to services.

COMMON NETWORK & DNS ISSUES



DNS FAILURES

Unable to resolve domain names.
Check DNS records, resolvers, and caching.



CONNECTION REFUSED

The target host is reachable, but the service is not accepting connections.



CONNECTION TIMEOUT

No response from the target host.
Check routing, firewalls, or service health.



PACKET LOSS

Packets are not reaching the destination consistently. Check network path.



PORT ACCESSIBILITY

Verify if a specific port is open and accessible from the source.



ROUTING FAILURES

Traffic is not taking the correct path.
Check routes, gateways, and peering.



TLS PROBLEMS

Certificate issues, handshake failures, or protocol incompatibilities.



LOAD BALANCER CONNECTIVITY

Health check failures, backend unreachable, or listener misconfiguration.



FIREWALL PROBLEMS

Traffic is blocked by security groups, NACLs, firewalls, or WAF.



INTERMITTENT NETWORK FAILURES

Issues that occur randomly. Check stability, MTU, congestion, or DNS TTL.

ESSENTIAL COMMANDS

```
$ dig example.com
```

Query DNS records and debug DNS resolution.

```
$ nslookup example.com
```

Test DNS resolution and nameserver.

```
$ curl -I https://example.com
```

Test HTTP/HTTPS connectivity.

```
$ ping example.com
```

Check basic connectivity and packet loss.

```
$ traceroute example.com
```

Trace the network path to the destination.

```
$ ss -tulpn
```

Show listening ports and active connections.

```
$ nc -vz example.com 443
```

Test port connectivity (TCP/UDP).

```
$ openssl s_client -connect  
example.com:443 -servername  
example.com
```

Test TLS handshake and certificate details.

QUICK CHECK FLOW



Identify Symptom



Test DNS Resolution



Test Network Path



Check Port Access



Verify Service Connectivity



TIP: Always test from multiple points (your workstation, jump host, and another AZ/VPC). Compare with a working system to identify where the path breaks.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

10. APPLICATION AND API INCIDENTS



Goal: Quickly identify and resolve application and API issues that impact functionality, performance, and availability.

COMMON APPLICATION & API ISSUES



HTTP 4XX AND 5XX ERRORS

4xx are client errors (bad request, auth, not found).
5xx are server errors indicating server-side issues.



APPLICATION CRASHES

The application stops unexpectedly.
Check crash logs, exit codes, and recent changes.



DEPENDENCY FAILURES

Downstream services, databases, caches, or third-party APIs are unavailable or slow.



REQUEST TIMEOUTS

Requests take too long and exceed the configured timeout.



THREAD EXHAUSTION

All threads are busy and new requests cannot be processed.



CONNECTION POOL EXHAUSTION

No available connections to the database or external service.



CONFIGURATION ERRORS

Incorrect or missing configuration causes unexpected behavior.



AUTHENTICATION FAILURES

Invalid tokens, expired sessions, or identity provider issues.



RATE LIMITING

Too many requests are blocked or throttled by the system or upstream.



LATENCY SPIKES

Response time increases suddenly, impacting user experience.



READING APPLICATION LOGS

Logs provide the most reliable clues.
Filter, correlate, and look for patterns.

TROUBLESHOOTING CHECKLIST

- ✓ Identify the affected service and endpoints.
- ✓ Check error rate, latency, and saturation.
- ✓ Correlate with recent deployments or changes.
- ✓ Check dependencies and downstream services.
- ✓ Examine application logs and error messages.
- ✓ Validate configuration and environment variables.
- ✓ Check resource usage (CPU, memory, threads).
- ✓ Confirm authentication and authorization flows.
- ✓ Verify rate limits and throttling policies.
- ✓ Mitigate or roll back if needed.
- ✓ Communicate updates and document actions.

USEFUL COMMANDS & EXAMPLES

COMMAND	PURPOSE
<code>curl -I https://api.example.com/health</code>	Check API health endpoint
<code>curl -v https://api.example.com/resource</code>	Verbose request, see headers & errors
<code>curl -w "\nTime: %{time_total}s\n"</code>	Measure request time
<code>grep -i error app.log tail -n 50</code>	Find recent errors in log
<code>grep -i exception app.log tail -n 50</code>	Find recent exceptions
<code>tail -f app.log</code>	Stream live application logs
<code>jq . error.json</code>	Pretty print JSON error
<code>kubectl logs -f deployment/app</code>	Stream logs in Kubernetes
<code>systemctl status app</code>	Check service status
<code>ps aux grep java</code>	Check application processes



TIP: Don't guess. Collect evidence, compare with a healthy system, and change one thing at a time.



11. KUBERNETES ON-CALL TROUBLESHOOTING



Goal: Quickly identify and resolve Kubernetes issues that impact workload health, availability, and performance.

COMMON KUBERNETES ISSUES



PENDING PODS

Pods are not scheduled due to insufficient resources, node selectors, or taints.



CRASHLOOPBACKOFF

The container starts but crashes repeatedly. Check logs and application errors.



IMAGEPULLBACKOFF

Kubernetes cannot pull the container image. Check image name, registry, and credentials.



OOMKILLED

The container was killed due to out-of-memory. Check memory limits and usage.



FAILED READINESS PROBES

Pod is running but not ready to receive traffic. Check readiness probe configuration.



FAILED LIVENESS PROBES

Kubelet is killing the container because the liveness probe is failing.



SERVICE CONNECTIVITY

Service or pod cannot reach other services or external endpoints.



DNS PROBLEMS

Pods cannot resolve DNS names. Check CoreDNS and DNS configuration.



NODE FAILURES

Nodes are NotReady, unreachable, or evicted due to issues.



RESOURCE PRESSURE

CPU, memory, or disk pressure affects pod scheduling and performance.



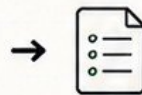
DEPLOYMENT FAILURES

Rollouts fail due to bad images, configuration, or failed health checks.

KUBERNETES TROUBLESHOOTING FLOW



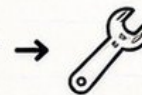
Identify Symptom



Gather Information



Determine Root Cause



Mitigate or Fix



Verify Recovery

ESSENTIAL KUBECTL COMMANDS

COMMAND	PURPOSE
<code>kubectl get pods</code>	List pods in the current namespace with status.
<code>kubectl describe pod <pod-name></code>	Get detailed information about a pod, including events.
<code>kubectl logs <pod-name></code>	View logs from the current container.
<code>kubectl logs <pod-name> --previous</code>	View logs from the previous container instance (if restarted).
<code>kubectl get events</code>	Show recent events in the namespace.
<code>kubectl top pods</code>	Show resource usage (CPU, Memory) for pods.
<code>kubectl top nodes</code>	Show resource usage for nodes.
<code>kubectl get endpoints</code>	Show endpoints for a service (IPs and ports).

QUICK CHECKS

- ✓ Check pod status and restarts
- ✓ Review pod events and logs
- ✓ Verify resource usage
- ✓ Validate probes and configuration
- ✓ Check service and DNS connectivity
- ✓ Confirm recent changes or deployments
- ✓ Inspect node health
- ✓ Correlate with monitoring and alerts



TIP: Always check events and logs first. They reveal the reason faster than guessing. Change one thing at a time and verify.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA



12. DATABASE, CACHE, AND STORAGE INCIDENTS



Goal: Quickly identify and resolve data layer issues that impact application performance, availability, and data integrity.

COMMON DATA LAYER INCIDENTS



DATABASE CONNECTION FAILURES

Applications cannot connect to the database. Check network, credentials, and database health.



SLOW QUERIES

Queries take too long to execute. Check query performance, indexes, and execution plans.



CONNECTION EXHAUSTION

Too many connections are open. New requests cannot get a connection. Check max connections and idle sessions.



LOCK CONTENTION

Transactions are blocked waiting for locks. Check long-running transactions and lock waits.



REPLICATION LAG

Replicas are behind the primary. Check replication status, delays, and I/O performance.



STORAGE EXHAUSTION

Disk is full or nearly full. Check disk usage, inode usage, and log growth.



CACHE FAILURES

Cache is down or returning errors. Check cache connectivity and evictions.



REDIS AVAILABILITY

Redis is unreachable or not responding. Check Redis health, memory, and persistence.



DATA CONSISTENCY CONCERNS

Data mismatch, stale reads, or inconsistent state observed. Verify and investigate before taking action.



WHEN NOT TO MODIFY PRODUCTION DATA

Avoid manual changes unless absolutely necessary. Risk of data corruption, inconsistency, and downtime.

USEFUL COMMANDS & TOOLS

COMMAND / TOOL	PURPOSE
<code>mysql -u user -p -e "show processlist;"</code>	View MySQL active connections
<code>psql -U user -c "\l"</code>	List PostgreSQL databases
<code>psql -U user -c "select * from pg_stat_activity;"</code>	View PostgreSQL active queries
<code>show engine innodb status \G</code>	Check InnoDB lock & transaction status
<code>pg_replication_slots</code>	Check replication slots
<code>df -h</code>	Check filesystem disk usage
<code>df -i</code>	Check inode usage
<code>redis-cli info</code>	Redis server information
<code>redis-cli -h host -p 6379 ping</code>	Test Redis connectivity
<code>aws rds describe-db-instances</code>	Check RDS instance status
<code>iostat -x 1</code>	Check disk I/O performance
<code>iostat -o</code>	See top I/O consuming processes

QUICK DIAGNOSTIC FLOW



1. Identify Symptom



2. Check Connections



3. Check Performance



4. Check Logs & Metrics



5. Mitigate or Escalate

BEST PRACTICES

- ✓ Always verify the impact and blast radius before acting.
- ✓ Use read-only queries for investigation.
- ✓ Check monitoring dashboards and alarms first.
- ✓ Correlate with recent deployments or schema changes.
- ✓ Avoid running heavy queries during an incident.
- ✓ Escalate to DBAs for complex issues.
- ✓ Document actions and findings in the incident channel.
- ✓ Verify recovery and monitor post-fix.



TIP: Data is critical. Be careful with manual changes. Investigate thoroughly, take backups, and get approval before modifying.



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA

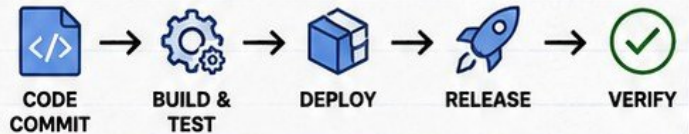


Telegram
@VERIQTA

DEPLOYMENT-RELATED INCIDENTS

- Failed deployments
- Bad releases
- Configuration changes
- Feature flag failures
- Database migration failures
- Partial rollouts
- Dependency version changes
- Canary failures
- Rollback versus roll-forward
- Verifying recovery after rollback

DEPLOYMENT PIPELINE



DEPLOYMENT INCIDENTS HAPPEN WHEN:

- A change introduces an unexpected failure.
- A release behaves differently in production.
- A rollout only partially completes.
- A rollback or recovery does not restore service.

COMMON CAUSES

-  Code defects
-  Misconfiguration
-  Database migration issues
-  Feature flag misconfiguration
-  Dependency incompatibility
-  Partial or failed rollouts

HOW TO DETECT

-  Error rate increase
-  Latency increase
-  Failed health checks
-  User complaints
-  Canary or rollout alerts
-  Deployment logs & events

IMPACT

-  User experience degradation
-  Revenue impact
-  Service downtime
-  Data inconsistency risk
-  Downstream service failures
-  Operational disruption

ROLLBACK VS ROLL-FORWARD

ROLLBACK

- ✓ Return to last known good version
- ✓ Fastest way to restore stability
- ✓ Use when the new release is unsafe



ROLL-FORWARD

- ✓ Fix the issue in the current version
- ✓ Maintain forward momentum
- ✓ Use when rollback is not feasible



VS

VERIFYING RECOVERY AFTER ROLLBACK

- ✓ Confirm the rollback completed successfully.
- ✓ Verify application health checks are passing.
- ✓ Monitor error rate and latency.
- ✓ Check logs for new or repeating errors.
- ✓ Validate critical user journeys.
- ✓ Confirm downstream dependencies are healthy.
- ✓ Monitor for at least one full business cycle.



TIP

Always prefer safe, reversible changes. Test in lower environments. Automate your rollbacks. Verify recovery from the user's perspective.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

INCIDENT MITIGATION AND RECOVERY



RESTORE SERVICE BEFORE FINDING PERFECTION

The goal is to make the system safe and stable, then improve.

MITIGATION AND RECOVERY ACTIONS

1.  **ROLLBACKS**
Return to the last known good version or configuration.
2.  **TRAFFIC SHIFTING**
Shift traffic away from the unhealthy version or region.
3.  **FEATURE DISABLING**
Turn off a risky feature or component quickly.
4.  **SCALING RESOURCES**
Add capacity to reduce pressure and stabilize the system.
5.  **RESTARTING SAFELY**
Restart services or pods carefully without causing more impact.
6.  **ISOLATING UNHEALTHY COMPONENTS**
Remove or quarantine the failing component.
7.  **FAILOVER**
Switch to a standby system or healthy environment.
8.  **DEGRADED OPERATION**
Keep core functionality running while non-essential features are limited.
9.  **TEMPORARY MITIGATIONS**
Apply short-term fixes to control impact while a permanent fix is prepared.
10.  **RECOVERY VERIFICATION**
Confirm the service is healthy and users are fully restored.
11.  **MONITOR CONTINUOUSLY**
Watch key metrics and alerts after recovery.

MITIGATION MINDSET



- ☒ Act fast, but stay safe.
- ☒ Change one thing at a time.
- ☒ Use the least risky option first.
- ☒ Prefer reversible changes.
- ☒ Communicate every action.
- ☒ Verify before declaring recovery.

MITIGATION FLOW



INCIDENT
DETECTED



ASSESS
IMPACT



APPLY
MITIGATION



VERIFY
RECOVERY



MONITOR
CONTINUOUSLY

EXAMPLES OF TEMPORARY MITIGATIONS

- ✓ Increase timeouts
- ✓ Disable non-critical integrations
- ✓ Reduce traffic to a single AZ or region
- ✓ Switch to a read replica
- ✓ Enable caching
- ✓ Increase resource limits
- ✓ Route traffic to a stable version



Temporary
actions are
acceptable.
Unstable systems
are not.

RECOVERY VERIFICATION CHECKLIST

- ☒ Key metrics are back to normal
- ☒ Synthetic checks are passing
- ☒ Error rates are within baseline
- ☒ Logs show no new critical errors
- ☒ Latency is within baseline
- ☒ Downstream dependencies are healthy
- ☒ User journeys are working
- ☒ Business impact is resolved



TIP

A good mitigation reduces impact quickly. A good engineer verifies fully before closing the incident.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

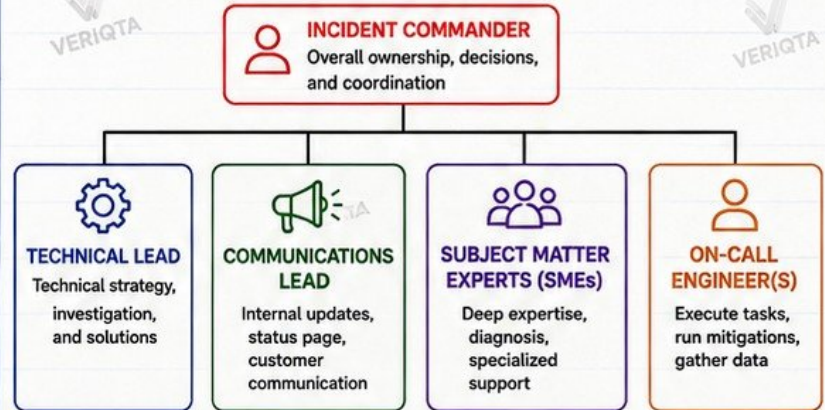
ESCALATION AND INCIDENT COMMAND



WHEN TO ESCALATE

- ✓ Impact is increasing or critical
- ✓ You lack the required expertise
- ✓ Multiple mitigation attempts failed
- ✓ Customer or business is at risk
- ✓ You are unsure of the next step
- ✓ Incident is growing in complexity
- ✓ Response is slower than expected
- ✓ Additional resources are needed
- ✓ Security or data integrity is threatened
- ✓ Escalate early, not late.

INCIDENT COMMAND STRUCTURE



CLEAR OWNERSHIP

- One person owns the incident. Others support.
- Avoid shared ownership. Avoid confusion.
- Everyone knows their role. Everyone knows who decides.
- Document decisions and important actions.

ESCALATION PATHS



WAR ROOMS

- ✓ Use a dedicated incident channel.
- ✓ Keep the room small and focused.
- ✓ Invite only necessary participants.
- ✓ Share information and updates in one place.
- ✓ Post timelines, decisions, and actions.
- ✓ Keep conversations clear and actionable.

PREVENTING TOO MANY ENGINEERS FROM MAKING CHANGES

- ✓ Too many cooks break production.
- ✓ Limit number of people with change access.
- ✓ One person applies the change.
- ✓ Others observe, validate, and advise.
- ✓ Use pair review for risky actions.
- ✓ Coordinate before touching production.
- ✓ Reduce noise. Increase focus.

DECISION AUTHORITY DURING INCIDENTS

INCIDENT COMMANDER	Has final say on trade-offs, priorities, and major decisions.
TECHNICAL LEAD	Recommends technical approach and mitigation plan.
COMMUNICATIONS LEAD	Owns all external and internal communications.
SMEs	Provide expert input and validate technical solutions.
ON-CALL ENGINEERS	Execute assigned actions and report results.

TIP

Strong incident command reduces chaos, improves speed, and protects customers and the business.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

INCIDENT COMMUNICATION

1. INTERNAL UPDATES



- Keep the response team aligned.
- Share facts, not assumptions.
- Use the incident channel.
- Encourage questions and clarity.

2. EXECUTIVE UPDATES



- Focus on business impact and risk.
- Avoid technical jargon.
- Highlight decisions, needs, and ETA.
- Keep updates short and strategic.

3. CUSTOMER-FACING COMMUNICATION



- Be honest, clear, and empathetic.
- Explain impact in simple terms.
- Set expectations.
- Update even if there is no new info.

4. STATUS PAGES



- Use a status page for public updates.
- Keep information consistent.
- Update regularly.
- Close the incident publicly.

5. COMMUNICATING UNCERTAINTY



- It is okay to say "We are investigating."
- Be transparent about what is known.
- Avoid guessing.
- Update as more information is known.

WHAT INFORMATION TO INCLUDE



- ✓ What happened
- ✓ When it started
- ✓ What systems or users are impacted
- ✓ The current status
- ✓ What caused it (if known)
- ✓ What actions are being taken
- ✓ What you are working on next
- ✓ When the next update will be provided
- ✓ When the incident is resolved

UPDATE FREQUENCY GUIDE



SEVERITY	UPDATE FREQUENCY
SEV1 (Critical)	Every 15–30 minutes
SEV2 (High)	Every 30–60 minutes
SEV3 (Medium)	Every 60–120 minutes
SEV4 (Low)	Every 2–4 hours

AVOIDING SPECULATION



- ✗ Do not guess the root cause.
- ✗ Do not promise what you cannot guarantee.
- ✗ Do not share unverified information.
- ✗ Do not blame teams or individuals.
- ✗ Communicate facts only.

MAINTAINING AN INCIDENT TIMELINE



ALERT
Record the alert time and source.



INVESTIGATION
Document key observations and initial findings.



ACTIONS
Log all actions taken with timestamps.



COMMUNICATION
Record updates sent to teams, executives, and customers.



RESOLUTION
Record the resolution time and final status.

COMMUNICATION PRINCIPLES

- ✓ Be clear, concise, and consistent.
- ✓ Communicate early and often.
- ✓ Empathy builds trust.
- ✓ Use language the audience can understand.
- ✓ Close the loop after resolution.

EXAMPLE UPDATE STRUCTURE



IMPACT
What is impacted and how severe is it?



CURRENT STATUS
What is the current state of the incident?



ACTIONS TAKEN
What actions have been taken so far?



NEXT STEP
What is happening next to resolve this?



NEXT UPDATE
When will the next update be provided?

**TIP**

Good communication reduces anxiety, builds trust, and helps everyone make better decisions during an incident.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

DANGEROUS ON-CALL MISTAKES



These mistakes can make incidents worse, extend downtime, and cause data loss or outages. Avoid them at all costs.

1

RESTARTING WITHOUT INVESTIGATION

You may restart the wrong thing and make the real issue harder to find.



2

DELETING RESOURCES DURING DIAGNOSIS

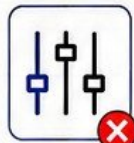
Deleting things can cause data loss and make recovery much more difficult.



3

MAKING MULTIPLE CHANGES SIMULTANEOUSLY

You won't know which change fixed the issue or caused a new problem.



4

IGNORING RECENT DEPLOYMENTS

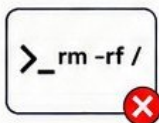
Recent changes are one of the most common causes of incidents. Always check first.



5

RUNNING DESTRUCTIVE COMMANDS

Commands like `rm -rf /`, `:(){:}&|;`, or mass updates can destroy systems or data.



6

CHANGING PRODUCTION WITHOUT RECORDING ACTIONS

If you don't document changes, you risk confusion, repeated actions, and bigger failures.



7

TREATING SYMPTOMS AS ROOT CAUSES

Fixing the symptom may hide the real problem and cause the issue to return.



8

WORKING ALONE TOO LONG

You can miss critical context or make blind decisions. Bring others in early.



9

FAILING TO ESCALATE

Escalate when the incident exceeds your expertise, time, or permissions.



10

DECLARING RECOVERY TOO EARLY

A service may look healthy but still be degraded. Always verify from the user's perspective.



SAFE ON-CALL HABITS



- ✓ Investigate before taking action.
- ✓ Change one thing at a time.
- ✓ Document every action you take.
- ✓ Ask for help early.
- ✓ Communicate clearly and often.
- ✓ Verify before closing the incident.



ASK YOURSELF BEFORE YOU ACT

- Do I understand the impact?
- Do I have enough evidence?
- Could this make things worse?
- Is there a safer alternative?
- Have I documented this step?
- Who can help me if this escalates?



TIP

Slow down to go fast. Safe actions restore service.
Rushed actions create bigger incidents.



YouTube
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta

18. FROM RECOVERY TO ROOT CAUSE

- 1 Confirm service recovery
- 2 Continue monitoring
- 3 Preserve logs and evidence
- 4 Build the incident timeline
- 5 Identify contributing factors
- 6 Root cause versus trigger
- 7 Five Whys
- 8 Identify detection gaps
- 9 Identify response gaps
- 10 Create follow-up actions

THE INVESTIGATION FLOW



ROOT CAUSE vs TRIGGER



TRIGGER

The event that started the incident.



ROOT CAUSE

The underlying issue that made the incident possible.

FIVE WHYS EXAMPLE

- WHY 1 Service was down.
 WHY 2 Database connection failed.
 WHY 3 Database was overloaded.
 WHY 4 Too many slow queries.
 WHY 5 Missing index on table.

ROOT CAUSE:

Missing index on table.

IDENTIFY GAPS



DETECTION GAPS

What failed to alert us early enough?



RESPONSE GAPS

What slowed down our response or recovery?



The goal is not just to fix today's incident, but to prevent tomorrow's. Every incident is an opportunity to make our systems and our teams stronger.

BLAMELESS
IMPROVEMENT
CONTINUOUS LEARNING



YouTube
@VERIQTA



GitHub
@VERIQTA



LinkedIn
@VERIQTA



X
@VERIQTA



Instagram
@VERIQTA



Telegram
@VERIQTA

19. POST-INCIDENT REVIEW

BLAMELESS POSTMORTEMS

- Focus on systems, not people.
- Blameless means learning, not fault finding.
- The goal is to improve our systems and prevent recurrence.

THE POST-INCIDENT REVIEW PROCESS



POST-INCIDENT REVIEW CONTENT

1 INCIDENT SUMMARY



- What happened
- When it happened
- Services affected
- Duration

2 CUSTOMER IMPACT



- Who was affected
- How many users
- Business impact
- SLA impact

3 TIMELINE



- Key events in order
- Timestamps
- Major decisions
- Milestones

4 DETECTION



- How was the issue detected?
- What alerts fired?
- Detection gaps

5 RESPONSE



- What actions were taken?
- Who did what?
- Communication steps

6 ROOT CAUSE



- The underlying cause that made the incident possible, not just the trigger.

7 CONTRIBUTING FACTORS



- What conditions or weaknesses contributed to the incident?

8 WHAT WORKED



- What went well?
- What helped us resolve or mitigate quickly?

9 WHAT FAILED



- What did not work?
- Where did we struggle?
- What slowed us down?

10 CORRECTIVE ACTIONS



- Actions to fix the root cause and contributing factors.

11 OWNERS & DEADLINES



- Who owns each action?
- Target completion date.

12 PREVENTING RECURRENCE



- How we prevent this from happening again.
- System improvements and process changes.

GOOD POSTMORTEM PRACTICES

- ☒ Write facts, not opinions.
- ☒ Include data and evidence.
- ☒ Keep it clear, concise, and complete.
- ☒ Review actions until they are completed.
- ☒ Share learnings across the organization.



REMEMBER

Every incident is an opportunity to make our systems stronger and our customers happier.

POSTMORTEM CHECKLIST

- ☒ All sections completed
- ☒ Root cause identified
- ☒ Actions defined
- ☒ Owners assigned
- ☒ Deadlines set
- ☒ Review and approval done
- ☒ Actions tracked to completion

20. ON-CALL PRODUCTION READINESS



BEFORE AN INCIDENT HAPPENS,
READINESS IS WHAT MAKES THE DIFFERENCE.

1

MONITORING READINESS



- Key services and dependencies are monitored
- Dashboards are up to date
- Metrics have meaningful baselines
- Synthetic checks are in place



2

ALERT READINESS



- Alerts are accurate and actionable
- Severity levels are correct
- No duplicate or noisy alerts
- Escalation paths are configured



3

RUNBOOK READINESS



- Runbooks are clear and updated
- Step-by-step instructions exist
- Troubleshooting guides available
- Mitigation steps are tested



4

ACCESS READINESS



- All required access is available
- Secrets and credentials accessible
- Access tools are working
- MFA and SSO are functional



5

ESCALATION READINESS



- Escalation matrix is clear
- On-call rotations are confirmed
- Contacts are reachable
- Escalation channels are tested



6

BACKUP & RECOVERY READINESS



- Backups are automated and verified
- Restore procedures are tested
- RPO and RTO are defined
- Recovery contacts are known



7

ROLLBACK READINESS



- Rollback procedures exist
- Previous known-good version is available
- Rollback process is tested
- Rollback decision criteria defined



8

DOCUMENTATION READINESS



- Architecture is documented
- Service ownership is clear
- Dependencies are documented
- System diagrams are updated



9

INCIDENT COMMUNICATION READINESS



- Internal communication channel ready
- Customer communication templates available
- Status page is configured
- Communication roles are defined



10

POSTMORTEM READINESS



- Postmortem template is available
- Storage location for reports is ready
- Review process is defined
- Action tracking process is in place



READINESS TODAY PREVENTS CHAOS TOMORROW.
PREPARATION IS RELIABILITY IN ACTION.



VERIQTA ON-CALL PRINCIPLES

- ✓ Be prepared.
- ✓ Be observant.
- ✓ Act with confidence.
- ✓ Communicate clearly.
- ✓ Escalate early.

- ✓ Protect customers.
- ✓ Learn from every incident.
- ✓ Document everything.
- ✓ Improve continuously.
- ✓ Make reliability a habit.

YOU ARE
THE CALM
IN THE CHAOS.
